



STARVNT CORE

DAY 3 EXECUTION PLAN

ADVANCED AUTOMATION, EVENT RELIABILITY & PRODUCTION FOUNDATION

Prepared for Keshab (Technical Lead / Full-Stack)

**Execution principle: BUILD → PUSH → CONNECT → BREAK → RECOVER → TEST
→ DEMONSTRATE**

**DAY 1 BUILT THE FOUNDATION. DAY 2 INTRODUCED AUTOMATION. DAY 3 PROVES WHETHER
THE FOUNDATION CAN SURVIVE REAL-WORLD BEHAVIOR.**

0 | DAY 3 EXECUTIVE DIRECTION

Day 1 established the StarVnt Core foundation and Customer Module. Day 2 established the Automation Foundation. Day 3 must now move beyond basic CRUD and prove that the Core can process real business events reliably, safely, and audibly.

Day 3 is not about creating another isolated architecture. It is about connecting and strengthening what already exists.

DO NOT CREATE DUPLICATE CUSTOMER FUNCTIONALITY.

Use and extend the existing:

StarVnt Core
↓
Customer Module
↓
Existing APIs
↓
Existing Database Structure
↓
Existing Automation Foundation

Primary objective: transform the current Core from a CRUD-oriented application into an event-aware operational platform that can detect, persist, process, automate, recover, audit, and display business activity.

The target mindset is:

BUILD
↓
CONNECT
↓
FAIL SAFELY
↓
RECOVER
↓
AUDIT
↓
DEMONSTRATE

1 | DAY 3 START CONDITION

Before writing new code, inspect the current repository and verify the actual state of previous work. Completion is based on working evidence, not on the existence of files.

A component is considered complete only when it is:

- Implemented
- Integrated into the existing application
- Tested
- Committed
- Pushed to GitHub
- Demonstrated with working proof

If a required Day 2 automation component is incomplete, finish that foundation first. Do not create a workaround or parallel implementation just to move forward.

DAY 1 CHECK

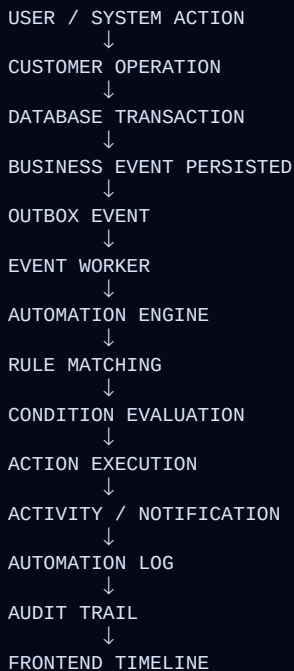
ITEM	STATUS
Frontend Running	YES / NO
Backend Running	YES / NO
MongoDB Connected	YES / NO
Customer API Working	YES / NO
Customer Data Stored	YES / NO
Frontend → Backend Working	YES / NO

DAY 2 CHECK

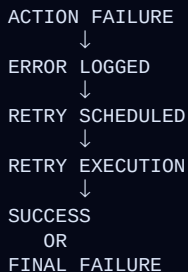
ITEM	STATUS
Event System Working	YES / NO
Event Dispatcher Working	YES / NO
Automation Service Working	YES / NO
Rule Evaluation Working	YES / NO
Condition Evaluation Working	YES / NO
Action Execution Working	YES / NO
Activity Foundation Working	YES / NO
Automation Logging Working	YES / NO

2 | DAY 3 PRIMARY MISSION

Build a reliable automation execution pipeline around the existing Customer module.



When something fails, the system must preserve state and make recovery explicit:



The objective is not maximum feature count. The objective is operational trustworthiness.

Day 3 success question:

Can StarVnt Core safely process a real business event when the system experiences duplicate delivery, temporary action failure, invalid requests, invalid state transitions, or unauthorized access?

3 | NO-DUPLICATE DEVELOPMENT RULE

Before creating anything, inspect the repository. Existing Customer infrastructure must be extended, not recreated.

Existing backend assets expected to be reused:

- Customer Model
- Customer Controller
- Customer Service
- Customer Routes
- Database Configuration
- Authentication Middleware
- Authorization Middleware
- Existing Event Logic
- Existing Automation Logic

Existing frontend assets expected to be reused:

- Customer List
- Customer Create Form
- Customer Details
- API Client
- Authentication Flow

The required engineering sequence is:

REVIEW
↓
REUSE
↓
EXTEND
↓
INTEGRATE
↓
TEST

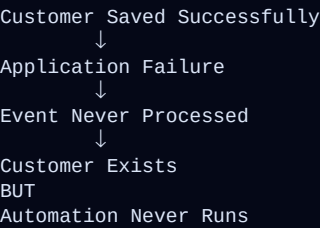
Do not create:

- A second Customer model
- A second Customer API
- A second database
- A separate automation demo project
- A duplicate Activity system
- A duplicate authentication system

If functionality already exists, strengthen it instead of replacing it.

4 | EVENT PERSISTENCE - TRANSACTIONAL OUTBOX FOUNDATION

An in-memory event such as emit("CUSTOMER_CREATED") is not enough for a business platform. A customer can be saved successfully and the application can then fail before the event is processed.



To avoid this reliability gap, introduce a persistent event / outbox structure.

ITEM	STATUS
eventId	Unique event identity
eventName	e.g. CUSTOMER_CREATED
entityType	e.g. CUSTOMER
entityId	Business entity identifier
payload	Structured event data
status	PENDING / PROCESSING / PROCESSED / FAILED
attempts	Current retry count
maxAttempts	Configured retry ceiling
availableAt	When event can be processed
processedAt	Successful completion timestamp
lastError	Most recent failure reason
createdAt / updatedAt	Persistence timestamps

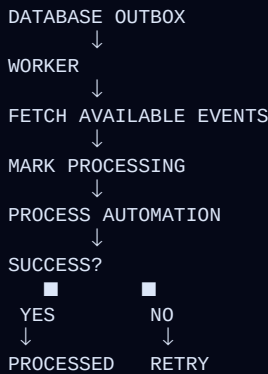
Critical rule:

The business event should only exist after the business operation has successfully committed. The outbox record must be part of the same reliable persistence boundary as

the business operation where the chosen database/transaction strategy permits it.

5 | EVENT WORKER / PROCESSOR

Create a simple background processing mechanism for pending events. Kafka, a distributed queue platform, or a microservice fleet is not required for V1.



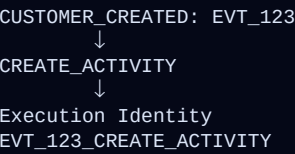
Worker responsibilities:

- Fetch available pending events
- Avoid simultaneous duplicate processing where practical
- Mark the event as processing before execution where appropriate
- Send the event into the Automation Engine
- Record success
- Record failure
- Schedule the next retry when allowed

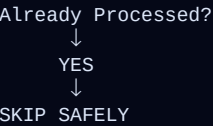
The worker should remain small and deterministic. The Automation Engine should own rule execution; the worker should own reliable event delivery and lifecycle management.

6 | IDEMPOTENCY - NON-NEGOTIABLE

Event processors can receive or process the same event more than once. The system must therefore make action execution idempotent.



Before executing an action, check whether that exact action has already completed successfully.



Required test:

ITEM	STATUS
Process CUSTOMER_CREATED EVT_123 first time	Exactly 1 Activity created
Process the same event a second time	No additional Activity
Review logs	Duplicate / skipped execution is traceable

Idempotency should be applied to every operational action that must not repeat accidentally, not just Activity creation.

7 | RETRY & FAILURE RECOVERY

A temporary failure in one automation action should not erase the business event or cause silent loss of work.

```
CUSTOMER_CREATED
↓
CREATE_ACTIVITY
↓
TEMPORARY_FAILURE
↓
LOG_FAILURE
↓
RETRY
↓
SUCCESS
OR
FINAL_FAILURE
```

Recommended V1 retry strategy:

```
Attempt 1
↓ fail
Attempt 2
↓ fail
Attempt 3
↓
SUCCESS / FINAL_FAILURE
```

Set a clear maximum retry count, e.g. `maxAttempts = 3`. Exponential backoff can be introduced later; V1 should remain simple and observable.

After the retry limit is reached:

```
Event Status = FAILED
Automation Log = FAILED
Last Error = Recorded
```

Never hide a failed automation. A failed event should remain discoverable for support, debugging, and future operational tooling.

8 | AUTOMATION EXECUTION LOG

Automation Log is operational evidence for what the automation system attempted. It should be separate from general application logs.

ITEM	STATUS
automationLogId	Unique execution log ID
eventId	Source event
ruleId	Rule that matched
entityType / entityId	Business target
actionType	Action executed
status	SUCCESS / FAILED / SKIPPED
attempt	Execution attempt number
idempotencyKey	Duplicate prevention identity
error	Failure details where relevant
startedAt / completedAt	Execution timing
createdAt	Log timestamp

The log must allow the StarVnt team to answer:

- What happened?
- Why did it happen?
- Which rule triggered?
- Which action executed?
- Did it succeed?
- Did it fail?
- Was it retried?
- Was it skipped because it was already processed?

Example:

Event: CUSTOMER_CREATED
Rule: CUSTOMER_CREATED_ACTIVITY_RULE
Action: CREATE_ACTIVITY
Status: SUCCESS
Attempt: 1

9 | CUSTOMER ACTIVITY TIMELINE - REAL OPERATIONAL HISTORY

Do not build a decorative timeline. Activity should become reusable operational infrastructure.

ITEM	STATUS
activityId	Unique activity identifier
organizationId	Tenant / organization scope where applicable
entityType	CUSTOMER / LEAD / ENQUIRY / etc.
entityId	Referenced business entity
actionType	Business activity type
message	Human-readable description
source	USER / SYSTEM / AUTOMATION / ADMIN
metadata	Optional structured context
createdBy	Actor when relevant
createdAt	Timestamp

Example operational history:

Customer Created
↓
Status Changed
↓
Note Added
↓
Follow-up Created
↓
Automation Executed
↓
Notification Generated

The same Activity architecture should later support:

CUSTOMER
LEAD
ENQUIRY
BOOKING
VENDOR
PAYMENT
EVENT

Build once. Reuse across StarVnt.

10 | SCHEDULED AUTOMATION FOUNDATION

Day 3 should include one real scheduled automation to prove that StarVnt can evaluate business conditions over time.

Recommended first use case:

FOLLOW-UP OVERDUE DETECTION

```
Follow-up Status = PENDING
AND
scheduledAt < current time
↓
FOLLOW_UP_OVERDUE
↓
Update Status
↓
Create Activity
↓
Create Notification / Alert
↓
Create Automation Log
```

Suggested status values:

```
PENDING
COMPLETED
OVERDUE
MISSED
```

One reliable scheduled job is enough for V1. Do not build a full scheduling platform or visual scheduler UI.

11 | DUPLICATE REMINDER PREVENTION

If a scheduler runs every hour, the same overdue follow-up must not generate the same notification every hour.

10:00 AM → Overdue detected → Alert created
11:00 AM → Already processed → SKIP
12:00 PM → Already processed → SKIP

Implement a deterministic duplicate check using a suitable idempotency strategy, for example:

FOLLOWUP_ID
+
AUTOMATION_TYPE
+
RELEVANT STATUS / VERSION

OR

AUTOMATION EXECUTION LOG LOOKUP

Required demonstration:

ITEM	STATUS
First scheduled evaluation	Alert / activity created
Second evaluation for same follow-up	No duplicate alert
Automation evidence	Skip or dedupe is traceable

12 | SECURITY - AUTHENTICATION IS NOT AUTHORIZATION

Being logged in does not automatically grant access to every resource. Day 3 must verify both authentication and authorization.

TEST A - Unauthenticated request

No Token
↓
Protected Customer API
↓
401 UNAUTHORIZED

TEST B - Authenticated but unauthorized

User A
↓
Authenticated
↓
Requests User B Customer
↓
403 FORBIDDEN
(or safe not-found strategy)

Server-side ownership rule:

Authenticated User
↓
Organization / Tenant
↓
Resource Ownership
↓
Permission
↓
ALLOW / DENY

Never treat customerId, userId, or organizationId submitted by the frontend as proof of authority. The server must derive and validate ownership.

13 | INPUT VALIDATION & API ERROR STANDARD

Before expanding automation, ensure the existing APIs reject malformed input consistently.

Customer validation should cover at minimum:

- Name
- Email
- Phone
- Status
- Source

Follow-up validation should cover at minimum:

- Customer ID
- Scheduled date
- Status
- Assigned user

Standard response example:

```
{
  "success": false,
  "code": "VALIDATION_ERROR",
  "message": "Invalid customer data"
}
```

Recommended error codes:

```
VALIDATION_ERROR
NOT_FOUND
UNAUTHORIZED
FORBIDDEN
DUPLICATE_RESOURCE
INVALID_STATE
INTERNAL_ERROR
AUTOMATION_FAILED
```

Do not expose stack traces, secrets, database internals, or sensitive implementation details to the frontend.

14 | STATE TRANSITION PROTECTION

Business status should not be freely mutable into arbitrary states. The Core should validate transitions.

NEW
↓
ACTIVE
↓
INACTIVE
↓
ARCHIVED

Before changing state:

Current Status
↓
Requested Status
↓
Transition Allowed?
■ ■
YES NO
↓ ↓
EXECUTE REJECT

Example:

ARCHIVED
↓
ACTIVE

REJECT unless explicitly allowed by the defined business state policy.

This state-transition concept should be reusable later for Leads, Quotes, Bookings, and Payments.

15 | AUDIT LOGGING

Automation Log and Audit Log answer different questions and should remain separate.

Automation Log: What did the automation system attempt or execute?

Audit Log: Who changed business data, what changed, and when?

ITEM	STATUS
auditId	Unique audit record
entityType / entityId	Affected business object
action	What changed
field	Changed field
previousValue	Before state
newValue	After state
changedBy	User / system actor
source	USER / SYSTEM / AUTOMATION / ADMIN
createdAt	Timestamp

Example:

Entity: CUSTOMER
Field: status
Before: ACTIVE
After: INACTIVE
Changed By: USER_123
Source: USER

Audit records will later support Admin Cockpit, operations, dispute resolution, compliance, and debugging.

16 | OBSERVABILITY

Add a minimum operational visibility layer so that the engineering team can determine whether Core is healthy without guessing.

Health endpoint:

```
GET /api/health
```

Minimum output:

```
{
  "status": "healthy",
  "database": "connected",
  "uptime": 12345
}
```

Important operations should be traceable with structured identifiers:

```
requestId
eventId
entityId
automationLogId
```

Example trace chain:

```
REQUEST → REQ_001
CUSTOMER → CUST_001
EVENT → EVT_001
AUTOMATION → AUTO_001
```

The objective is not to build a complete observability platform. The objective is to make failures diagnosable.

17 | REQUIRED DAY 3 TESTING

Manual testing alone is not sufficient. Day 3 should include automated or repeatable tests for the following critical behaviors.

ITEM	STATUS
1. Customer creation	Customer created successfully
2. Business event	CUSTOMER_CREATED outbox event exists
3. Event worker	Pending event is processed
4. Automation	Rule matches and action executes
5. Activity	Exactly one activity created
6. Idempotency	Same event produces no duplicate action
7. Failure	Action failure is logged and retry scheduled
8. Retry	Retry executes correctly
9. Maximum retries	Final state becomes FAILED
10. Overdue follow-up	OVERDUE + activity + notification/alert
11. Duplicate reminder	Second scheduler run creates no duplicate
12. Unauthenticated access	401
13. Unauthorized access	403 / safe denial
14. Invalid state transition	INVALID_STATE / rejection

Evidence should include test output or a reproducible demonstration, not just a written claim.

18 | DAY 3 GOLDEN DEMONSTRATION

The final demonstration should prove the normal path, failure recovery, idempotency, security, and scheduled automation.

DEMO 1 - NORMAL FLOW

```
Frontend
↓
Create Customer
↓
Backend API
↓
MongoDB
↓
Outbox Event
↓
Worker
↓
Automation Engine
↓
Rule
↓
Condition
↓
Action
↓
Activity
↓
Automation Log
↓
Frontend Timeline
```

DEMO 2 - FAILURE RECOVERY

```
Automation Action
↓
Temporary Failure
↓
Failure Logged
↓
Retry
↓
Success
```

DEMO 3 - IDEMPOTENCY

```
Same Event Processed Again
↓
Duplicate Detected
↓
Action Skipped
↓
No Duplicate Activity
```

DEMO 4 - SECURITY

```
Unauthorized User
↓
Protected Customer
↓
Access Denied
```

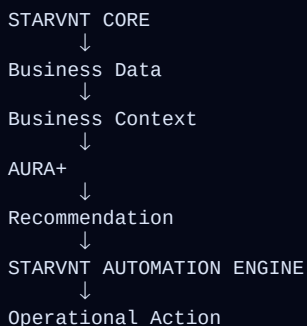
DEMO 5 - SCHEDULED AUTOMATION

```
Pending Follow-up
↓
Scheduled Time Passed
↓
Scheduler Detects
↓
OVERDUE
```

↓
Activity + Notification
↓
No Duplicate Reminder

19 | FUTURE AURA+ INTEGRATION BOUNDARY

Do not put Aura+ intelligence inside StarVnt Core. Keep business operations and intelligence separated.



Future recommendation example:

```
{
  "source": "AURA_PLUS",
  "recommendationType": "FOLLOW_UP",
  "entityType": "CUSTOMER",
  "entityId": "CUST123",
  "priority": "HIGH",
  "confidence": 0.91
}
```

Future Automation Engine checks:

```
Is Source Trusted?
↓
Is Recommendation Allowed?
↓
Is Automation Rule Enabled?
↓
Are Conditions Met?
↓
Execute Action
```

Aura+ recommends. Automation decides. Core executes business operations.

20 | DAY 3 FINAL DELIVERABLES

By the end of Day 3, the repository should contain a coherent reliability layer around the existing Customer and Automation foundations.

EVENT SYSTEM

ITEM	STATUS
Persistent Outbox	YES / NO
Event Worker	YES / NO
Event Status Tracking	YES / NO

AUTOMATION

ITEM	STATUS
Rule Evaluation	YES / NO
Condition Evaluation	YES / NO
Action Execution	YES / NO

RELIABILITY

ITEM	STATUS
Idempotency	YES / NO
Retry	YES / NO
Failure Recovery	YES / NO

ACTIVITY

ITEM	STATUS
Reusable Activity Model	YES / NO
Customer Timeline Integration	YES / NO

SCHEDULER

ITEM	STATUS
One Working Scheduled Automation	YES / NO
Overdue Detection	YES / NO
Duplicate Prevention	YES / NO

SECURITY

ITEM	STATUS
Authentication Test	YES / NO
Authorization Test	YES / NO
Ownership / Tenant Validation	YES / NO

DATA QUALITY

ITEM	STATUS
Input Validation	YES / NO
Standard API Errors	YES / NO
State Transition Protection	YES / NO

AUDITABILITY

ITEM	STATUS
Automation Log	YES / NO
Audit Log	YES / NO

OBSERVABILITY

ITEM	STATUS
Health Endpoint	YES / NO
Structured Operational Logging	YES / NO

21 | DAY 3 FINAL REPORT FORMAT

Submit the following report with actual values and links. Do not replace evidence with general statements.

STARVNT CORE - DAY 3 ADVANCED REPORT

CORE STATUS

Frontend: YES / NO
Backend: YES / NO
MongoDB: YES / NO
Health Endpoint: YES / NO

EVENT SYSTEM

CUSTOMER_CREATED: YES / NO
Outbox Persistence: YES / NO
Worker: YES / NO
Event Status Tracking: YES / NO

AUTOMATION

Rule Engine: YES / NO
Condition Engine: YES / NO
Action Engine: YES / NO
Automation Log: YES / NO

RELIABILITY

Idempotency: YES / NO
Duplicate Prevention Tested: YES / NO
Retry: YES / NO
Failure Recovery: YES / NO
Max Retry Handling: YES / NO

ACTIVITY SYSTEM

Activity Creation: YES / NO
Database Storage: YES / NO
Customer Timeline API: YES / NO
Frontend Timeline: YES / NO

SCHEDULER

Scheduled Job: YES / NO
Follow-up Overdue Detection: YES / NO
Duplicate Reminder Prevention: YES / NO

SECURITY

Authentication: YES / NO
Authorization: YES / NO
Ownership Check: YES / NO
Unauthorized Access Tested: YES / NO

DATA PROTECTION

Validation: YES / NO
API Error Standardization: YES / NO
Invalid State Protection: YES / NO

AUDIT

Automation Log: YES / NO
Audit Log: YES / NO

TESTING

Functional Tests: YES / NO
API Tests: YES / NO
Idempotency Tests: YES / NO
Retry Tests: YES / NO
Failure Tests: YES / NO
Security Tests: YES / NO

END-TO-END DEMO

Customer Created
↓
Database
↓
Outbox Event
↓
Worker
↓

Automation
↓
Action
↓
Activity
↓
Timeline

STATUS: WORKING / PARTIAL / NOT WORKING

GITHUB

Repository:

Branch:

Latest Commit:

WORKING PROOF

API Proof:

Database Proof:

Automation Proof:

Timeline Proof:

Test Proof:

BLOCKERS

- 1.
- 2.

DAY 4 RECOMMENDATION

- 1.
- 2.

22 | DAY 3 ACCEPTANCE STANDARD

Day 3 is not complete because a folder was created, an API exists, a worker file was written, or a screenshot was produced.

The system must demonstrate:

BUSINESS ACTION
↓
PERSISTED EVENT
↓
RELIABLE PROCESSING
↓
AUTOMATION
↓
OPERATIONAL ACTION
↓
AUDIT
↓
FAILURE RECOVERY
↓
SAFE FRONTEND RESULT

The platform should remain safe when confronted with:

DUPLICATE EVENT
FAILED ACTION
UNAUTHORIZED REQUEST
INVALID INPUT
INVALID STATE TRANSITION
SCHEDULER RE-RUN

Strong success means the team can prove the system works, fails safely, recovers predictably, and leaves an auditable trail.

23 | FINAL INSTRUCTION TO KESHAB

Keshab, Day 3 is not about building more software simply to increase the feature count. It is about making the software we already have trustworthy enough to build on.

Do not rebuild the Customer Module. Do not create a separate automation demo. Do not introduce unnecessary architecture.

Instead:

REVIEW WHAT EXISTS
↓
REUSE WHAT EXISTS
↓
STRENGTHEN WHAT EXISTS
↓
CONNECT WHAT EXISTS
↓
TEST FAILURE
↓
PROVE RECOVERY

Our engineering standard:

BUILD → PUSH → CONNECT → BREAK → RECOVER → TEST → DEMONSTRATE

By the end of Day 3, StarVnt Core should demonstrate that it can:

- Capture real business events
- Persist events safely
- Process automation reliably
- Prevent duplicate actions
- Recover from temporary failures
- Protect customer data
- Track operational history
- Maintain auditability
- Detect scheduled business conditions
- Display real results in the application

DAY 3 = RELIABILITY + AUTOMATION + SECURITY + AUDIT + REAL WORKING PROOF

StarVnt Core should not become a collection of disconnected modules. We are building one connected business ecosystem - where every action can create a signal, every signal can be processed safely, and every automated decision leaves a trace.

Build it clean. Build it once. Build it strong enough for what comes next. □

24 | STRATEGIC NEXT STEP - DAY 4

After Day 3, the next logical move is not another random feature. The foundation should now move into a practical Customer Tracking V1 operational layer.

Recommended Day 4 focus:

```
CUSTOMER FOUNDATION
+
AUTOMATION FOUNDATION
+
ACTIVITY TIMELINE
↓
CUSTOMER TRACKING V1
↓
NOTES + FOLLOW-UPS + TIMELINE + STATUS MOVEMENT
```

Why this is the logical next step:

- Customer foundation already exists
- Automation foundation can react to operational events
- Activity Timeline provides historical context
- Notes add operational memory
- Follow-ups introduce time-based business workflow
- Status movement creates a clear customer lifecycle

The principle remains the same:

Each day should make the previous day more valuable.

```
FOUNDATION
↓
AUTOMATION
↓
RELIABILITY
↓
CUSTOMER OPERATIONS
↓
INTELLIGENCE
↓
SCALE
```

This is how the StarVnt Core becomes a connected operating system rather than a collection of screens.